

Phase 4

LTE Module

AT-Command Bring-up

Manual cellular network connection & TCP echo verification

STATUS: COMPLETED on 2026-04-28. The T-A7608 4G modem boots, attaches to Vodafone Egypt with excellent signal (CSQ 22/31), opens a TCP socket to tcpbin.com:4242 over 4G, and successfully sends/echoes data end-to-end.

This document covers what was built, every AT command used and what it means, the working sequence in order, and the lessons learned during debugging. Phase 4 is the foundation for Phase 5 (where the STM32 takes over from the PC and drives the LTE module automatically).

1. Goal of Phase 4

Confirm that the T-A7608SA-H 4G LTE module can:

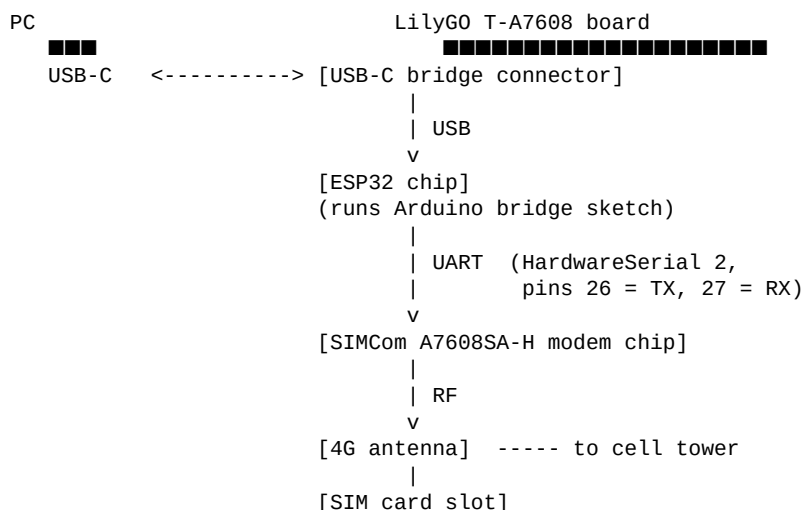
- Power on, boot, and respond to basic AT commands.
- Detect the SIM card and read its status.
- Register on the Vodafone Egypt cellular network with adequate signal strength.
- Attach to the GPRS / packet data service.
- Open a TCP socket to a public internet server over 4G.
- Send bytes over that socket and receive an echo back.

All of this is done **by typing AT commands manually** from the PC. The STM32 is NOT involved in Phase 4. We're just learning the modem's command set and verifying the network works before automating it in Phase 5.

2. Hardware Architecture

2.1 LilyGO T-A7608 board internals

The user's LTE module is a LilyGO T-A7608 development board, which contains **two chips working together**:



The ESP32 sits between the PC and the actual 4G modem. It handles the USB connection on one side and forwards bytes to/from the modem's UART on the other. Its job is just «translator» — it doesn't process the AT commands itself.

2.2 ESP32 control pins for the modem

ESP32 GPIO	Modem signal	Purpose
12	POWER_ON	Set HIGH to enable the 4G modem's power supply
4	PWRKEY	Pulse low for ~1 second to virtually press the modem's power button
5	RST	Hold HIGH to keep the modem out of reset
26	MODEM_RX	ESP32 sends from this pin → goes into the modem's RX
27	MODEM_TX	Modem sends from its TX → comes into the ESP32's pin 27

Without the ESP32 driving these pins, the 4G modem stays off — even though the USB connection appears to work. This was one of the first issues the user hit (see Section 6 lessons learned).

2.3 Power requirement

The 4G modem can draw bursts of up to 2 A during transmit. USB-C from a laptop typically only supplies 500 mA. For reliable operation, the LilyGO board needs additional power:

Option	How	Status
18650 Li-ion battery	Insert charged 18650 cell into the holder on the board	Best — recommended for production
External 5 V supply	Feed 5 V to the board's VBUS pin	Works
USB-C only	Plug board directly into laptop / phone charger	Works for AT testing, marginal for transmit bursts

3. Arduino Bridge Sketch

The ESP32 runs a small Arduino program that does two things at startup:

(1) drives the modem power-on sequence using GPIO pins, and (2) becomes a transparent bridge that forwards every byte between USB Serial and the modem UART in both directions.

3.1 Final working sketch

```
#include <HardwareSerial.h>

HardwareSerial gsmSerial(2);

#define MODEM_RX_PIN      27
#define MODEM_TX_PIN      26
#define MODEM_PWRKEY_PIN  4
#define MODEM_POWERON_PIN 12
#define MODEM_RST_PIN     5
#define PC_BAUD            115200
#define MODEM_BAUD         115200

void setup() {
  Serial.begin(PC_BAUD);
  delay(500);
  Serial.println("\n[ESP32] booting...");

  // Enable 4G modem power
  pinMode(MODEM_POWERON_PIN, OUTPUT);
  digitalWrite(MODEM_POWERON_PIN, HIGH);

  // Keep reset HIGH (not in reset)
  pinMode(MODEM_RST_PIN, OUTPUT);
  digitalWrite(MODEM_RST_PIN, HIGH);

  // Pulse PWRKEY to power on the modem
  pinMode(MODEM_PWRKEY_PIN, OUTPUT);
  digitalWrite(MODEM_PWRKEY_PIN, HIGH);
  delay(100);
  digitalWrite(MODEM_PWRKEY_PIN, LOW);
  delay(1000);
  digitalWrite(MODEM_PWRKEY_PIN, HIGH);

  Serial.println("[ESP32] PWRKEY pulsed, waiting for modem to boot...");
  delay(5000);

  gsmSerial.begin(MODEM_BAUD, SERIAL_8N1, MODEM_RX_PIN, MODEM_TX_PIN);

  Serial.println("[ESP32] Bridge ready. Type AT commands.");
}

void loop() {
  // PC -> modem (transparent passthrough)
  while (Serial.available() > 0) {
    gsmSerial.write((char)Serial.read());
  }

  // Modem -> PC (transparent passthrough)
  while (gsmSerial.available() > 0) {
    Serial.write((char)gsmSerial.read());
  }
}
```

3.2 Arduino IDE settings

Setting	Value
Board	ESP32 Dev Module
Upload Speed	921600

Setting	Value
CPU Frequency	240 MHz (WiFi/BT)
Flash Frequency	80 MHz
Flash Mode	QIO
Flash Size	4MB (32Mb)
Partition Scheme	Default 4MB with spiiffs
Port	whatever COM port the CH9102 driver assigns

3.3 CH9102 driver

The LilyGO board's USB bridge uses a **CH9102** chip (similar to CH340). Windows does not always install the driver automatically. If the COM port does not appear in Device Manager, download the driver from *wch-ic.com* (search «CH9102 Windows driver»), install it, and replug the USB-C cable.

3.4 Serial Monitor settings

Setting	Value
Baud rate	115200 (this is PC ↔ ESP32, not ESP32 ↔ modem)
Line ending	Both NL & CR (CRITICAL — modem ignores AT without proper line ending)

4. AT Command Reference

Every command typed during Phase 4, what it does, what reply to expect, and why it matters. Run them in the order shown in Section 5.

4.1 — AT

Definition: The simplest possible AT command. Comes from the original Hayes modem standard from the 1980s. Just two letters and Enter — used to verify that the modem is alive and listening.

Reply: OK

Why we use it: sanity check. If AT doesn't reply OK, no other command will work either. Always type this first.

4.2 — ATE0

Definition: Echo Off. Tells the modem «don't echo my commands back to me».

Reply: OK

Why we use it: by default the modem echoes every character you type. When the responses come, your output looks like:

AT+CSQ <-- echoed back

+CSQ: 22,99 <-- actual response

OK

After ATE0 the echo stops, so you only see the response — cleaner output.

4.3 — AT+CPIN?

Definition: «PIN?» Query the SIM card's lock status. Asks: «Is the SIM card present and unlocked?»

Replies:

+CPIN: READY — SIM is unlocked and ready (good)

+CPIN: SIM PIN — SIM needs a PIN code (use AT+CPIN="1234" to enter)

+CME ERROR: 10 — no SIM card inserted

Why we use it: if SIM isn't ready, all later network commands will fail. Confirm READY before continuing.

4.4 — AT+CSQ

Definition: «Cellular Signal Quality». Asks the modem how strong the signal from the cell tower is.

Reply: +CSQ: <rss>,<ber>

Interpretation:

rsi value	Signal quality
0–9	Marginal — connection unstable
10–14	OK — usable
15–19	Good
20–30	Excellent
31	Maximum (rare)
99	Unknown / not detectable

The second value (BER, Bit Error Rate) is usually 99 (unknown) on most modems. Ignore it. Only the first number matters. The user got **22,99** which is excellent.

4.5 — AT+CREG?

Definition: «Cellular network REGistration». Asks: «Am I registered with a cell tower right now?»

Reply: +CREG: <n>,<stat>

Interpretation of stat:

stat	Meaning
0	Not registered, not searching
1	Registered, on home network ✓
2	Searching for a network
3	Registration denied
4	Unknown
5	Registered, on roaming network ✓

Stat = 1 or 5 means you can use the network. Anything else and you cannot make data connections. The user got **0,1** = registered on home network (Vodafone Egypt) ✓.

4.6 — AT+CGATT?

Definition: «Cellular GPRS ATtach status». Asks: «Am I attached to the data (GPRS/4G) network, separate from voice?»

Replies:

+CGATT: 1 — attached to data network ✓

+CGATT: 0 — not attached. Use AT+CGATT=1 to attach.

Why: registration (CREG) is voice-side; GPRS attach (CGATT) is data-side. Both must be 1 before you can open TCP sockets.

4.7 — AT+CGDCONT

Definition: «Cellular GPRS Define CONText». Sets the APN (Access Point Name) — the gateway that lets your modem reach the public internet through the carrier.

Syntax: AT+CGDCONT=<cid>,"<type>","<apn>"

Example used: AT+CGDCONT=1,"IP","internet.vodafone.net"

Reply: OK

Parameters:

cid = context ID, almost always 1

type = "IP" for IPv4 (the standard choice)

apn = the carrier's APN string

Egyptian carrier	APN string
Vodafone (the user's SIM)	internet.vodafone.net
Orange	internet.orange
Etisalat	etisalat
WE	internet

4.8 — AT+CGACT

Definition: «Cellular GPRS ACTivate». Activates the data session you defined with CGDCONT. After this command, the modem has an IP address from the carrier and can reach the internet.

Syntax: AT+CGACT=<state>,<cid>

Used: AT+CGACT=1,1 — state=1 (activate), cid=1 (context 1)

Reply: OK (may take 5–10 seconds)

Why: CGDCONT only DEFINES the parameters; CGACT actually ESTABLISHES the connection. Without CGACT the modem has the APN string saved but no IP.

4.9 — AT+NETOPEN

Definition: Open the IP application stack inside the modem. Newer SIMCom firmware (A7608, A7670) uses this command set instead of the older AT+CIPMUX / AT+CIPSTART set found in SIM800 era.

Reply: OK followed by +NETOPEN: 0

Why: after CGACT the modem has IP connectivity, but the IP stack inside the modem isn't running yet. NETOPEN starts it so you can open TCP/UDP sockets.

4.10 — AT+CIOPEN

Definition: Open a TCP or UDP socket. Equivalent to a programming connect() call.

Syntax: AT+CIOPEN=<link_id>,"TCP",<host>,<port>

Used: AT+CIOPEN=0,"TCP","tcpbin.com",4242

Reply: OK followed by +CIOPEN: 0,0 (link 0, result 0 = success)

Why: tcpbin.com:4242 is a public test server that echoes back anything you send to it. Useful for verifying TCP works without setting up your own server.

Other result codes: 1=resolution fail (DNS), 2=connect fail (no route), 3=connect fail (refused), 4=socket allocation fail.

4.11 — AT+CIPSEND

Definition: Send bytes through an open socket.

Syntax: AT+CIPSEND=<link_id>,<byte_count>

Used: AT+CIPSEND=0,5 — send 5 bytes through link 0

Behaviour:

1. Modem replies with a > prompt
2. You type exactly <byte_count> characters (no Enter)
3. Modem replies +CIPSEND: 0,5,5 (link, requested, actual)
4. If the server echoes back, you'll see +IPD <n> followed by the data

4.12 — Unsolicited Result Codes (URCs)

These are messages the modem sends without you asking. They appear at any time in the terminal:

URC	Meaning
+CPIN: READY	SIM finished initializing (sent automatically at boot)
SMS Ready	SMS subsystem is ready
PB DONE	Phonebook service initialized
+IPD <n>	Incoming <n> bytes of data on a socket — followed by the actual bytes
+IPCLOSE: 0,1	Socket 0 was closed by the remote (1=closed by peer, 2=timeout)
+CFUN: 1	Modem entered full functionality mode

5. Working Sequence — In Order

These are the exact commands the user typed and the actual responses received during the successful Phase 4 run. Type each one, wait for the response, then proceed to the next.

5.1 — Verify modem is alive

```
> AT
OK

> ATE0
OK
```

5.2 — SIM and signal

```
> AT+CPIN?
+CPIN: READY
OK

> AT+CSQ
+CSQ: 22,99
OK

> AT+CREG?
+CREG: 0,1
OK
```

Signal of 22 is excellent. Stat = 1 = registered on Vodafone home network.

5.3 — APN and data session activation

```
> AT+CGDCONT=1,"IP","internet.vodafone.net"
OK

> AT+CGACT=1,1
OK
```

5.4 — Open IP stack

```
> AT+NETOPEN
OK
+NETOPEN: 0
```

5.5 — Open TCP socket

```
> AT+CIOPEN=0,"TCP","tcpbin.com",4242
OK
+CIOPEN: 0,0
```

0,0 = link 0 opened successfully. The TCP connection is now live.

5.6 — Send data

```
> AT+CIPSEND=0,5
>
(now type the 5 characters with no Enter:)
hello

(modem replies:)
OK
+CIPSEND: 0,5,5

(then the server echoes back the 5 bytes:)
+IPD 5
hello

(eventually the test server closes the connection:)
+IPCLOSE: 0,1
```

PHASE 4 COMPLETE. The full data path is verified end-to-end: PC → ESP32 bridge → A7608 modem → cell tower → 4G core → internet → tcpbin.com → back. The modem can connect to TCP services on the public internet.

6. Lessons Learned (from the actual debug session)

Phase 4 hit several roadblocks that took an hour or more to solve. They are documented here so the same time isn't wasted next time.

6.1 — «BRIDGE READY» appeared but AT got no response

Symptom: the ESP32 boot text printed «[*] BRIDGE READY», but typing AT in PuTTY did nothing.

Cause: «BRIDGE READY» only means the ESP32's bridge firmware started. The actual 4G modem chip is a separate IC and needs the ESP32 to drive its POWERON / PWRKEY pins to power on. The original sketch did not do this.

Fix: add the power-on sequence in setup() — set MODEM_POWERON_PIN (GPIO 12) HIGH, set MODEM_RST_PIN (GPIO 5) HIGH, pulse MODEM_PWRKEY_PIN (GPIO 4) low for 1 second, then wait 5 seconds before opening the modem UART.

6.2 — Garbled characters in Serial Monitor

Symptom: after fixing the power-on, the bridge produced unreadable «b'@\$@bbA`bb...» gibberish.

Cause: baud rate mismatch between ESP32 and modem.

Fix: the user's specific A7608 firmware uses 115200 baud (the standard default for SIMCom). Some boards report needing 921600 — try 115200 first, then 921600 if garbled.

Note: the PC-to-ESP32 baud (Serial Monitor) is independent and stays at 115200 regardless.

6.3 — Could not type in PuTTY

Symptom: keyboard input didn't appear in the terminal.

Cause: default PuTTY has Local Echo off, so characters only appear after the modem echoes them. If the modem isn't answering, typing looks invisible.

Fix: Right-click the PuTTY title bar → Change Settings → Terminal → **Local echo: Force on, Local line editing: Force on**. Or use the Arduino IDE Serial Monitor instead, which always shows your input.

6.4 — «NO PORTS DISCOVERED» in Arduino IDE

Symptom: Arduino IDE didn't show the LilyGO board's COM port.

Cause: the LilyGO board uses a CH9102 USB-serial chip (not the more common CH340 or CP2102). Windows doesn't have the driver built in.

Fix: install the CH9102 driver from wch-ic.com, replug the cable.

6.5 — AT+CIPMUX and AT+CIPSTART returned ERROR

Symptom: after CGACT succeeded, attempts to use the older IP commands (AT+CIPMUX=0 then AT+CIPSTART) both returned ERROR.

Cause: the A7608 firmware uses a newer IP command set that doesn't include CIPMUX/CIPSTART. The older syntax is from the SIM800/SIM900 era.

Fix: use AT+NETOPEN followed by AT+CIOPEN=0,"TCP",... and AT+CIPSEND=0,<n>. The link ID parameter is always required (0 for the first connection).

6.6 — One of the two T-A7608 modules didn't respond

Symptom: the second T-A7608 module didn't respond to AT even with the same cable and PC.

Cause: likely a hardware defect or different firmware. Could not be diagnosed remotely without comparing the two boards side by side.

Action: Phase 4 development continues on the working module. The broken one needs to be inspected (antenna connection, button labels, comparison with the working board) before Phase 6 (where two devices need to talk).

6.7 — Common typos to watch for

Wrong	Right
AT+CGADCONT	AT+CGDCONT (no extra A)
AT+CSPN?	AT+CPIN? (P-I-N like the SIM PIN)
AT+CIPMUX	AT+NETOPEN (CIPMUX is the old SIM800 command, A7608 uses NETOPEN)
AT+CIPSTART	AT+CIOPEN (same reason)

7. What's Next

Phase 4 proved that a human can connect to the cellular network, attach to data, and open a TCP socket through manual AT commands. Phase 5 puts the STM32 in charge of doing the same thing automatically.

7.1 — Phase 5: STM32 drives the LTE module

- Disconnect the LTE module from the ESP32 / USB-C bridge.
- Wire the LTE module's TXD/RXD pins directly to the STM32's USART1 (PA9, PA10).
- Power the LTE module from a stable 5 V supply (NOT the STM32's 3.3 V regulator).
- Common ground between LTE module, STM32, and any external supply.
- Write STM32 firmware that automates the AT sequence from Section 5: AT, ATE0, AT+CPIN?, AT+CSQ, AT+CREG?, AT+CGDCONT, AT+CGACT, AT+NETOPEN, AT+CIOPEN to the relay server.
- Start running the small Python relay server on the developer's laptop with ngrok exposing it.
- Replace tcpbin.com:4242 with the ngrok-provided host:port.
- STM32 sends "REG <phone_number>" to the relay and waits for "OK".

7.2 — Phase 6 onwards

Phase	Adds
6 — Two-node plain audio	Both devices connected. CALL/ANS protocol. Audio bytes flow through relay. No encryption yet.
7 — Hybrid encryption	mbedTLS RSA-512 wraps an AES-128 key at session start; AES-CTR encrypts bulk audio.
8 — Polish & demo	Physical Call/Answer buttons, status LEDs (idle/ringing/in-call), video backup of the demo.

Phase 4 is the gateway between «the device works alone on a desk» (Phases 1–3) and «two devices talk to each other over the internet» (Phase 5+). All the protocol learning happens here so Phase 5 firmware doesn't have to debug the modem in addition to its own logic.

End of Phase 4 document.